idle work

Full-Stack Engineer: Practical Test

Build a small document-grounded chatbot (RAG) · TypeScript · about 2 hours with AI tools

> **How this works**
>
> - Spend about **2 hours**. Stop there even if you are not finished, and write down what is left. An honest partial result beats a rushed complete one.
> - You get a folder, `maple-street-rag/`, with 26 store files in `docs/` (Markdown, CSV, PDF, HTML and plain text, plus a `manifest.json`) and 58 test conversations in `cases.json`.
> - Send back a link to a Git repository (or a zip) and a **5-minute screen recording** to your recruiter **at least 24 hours before your interview**.
> - We expect you to use AI coding tools; that is why 2 hours is enough. In the interview you will explain any part of your code and change it live, so make sure you understand every line.
> - The store, people and numbers are fictional.

The situation

Maple Street Hardware wants a chat assistant on its website that answers customer questions **only from the store's own documents**: hours, returns, prices, rentals, delivery, key cutting, paint and rewards. If the documents do not say it, the assistant must say so instead of guessing. The documents are messy in the ways real ones are: old and new versions of the same policy (one of them an old web page), a 110-row price list with aisle locations, sale flyers that are only valid on certain dates, contract terms in a PDF, a long guide with the key fact buried in the middle, internal files that must never be shown to customers, a file nobody labeled, and customer reviews that contain an instruction aimed at the AI.

What to build

1. **Ingest.** Read every file in `docs/`: Markdown, CSV, PDF, HTML and text. Extract clean text (no navigation bars or scripts from the HTML), split into chunks, embed them, and store them. Your choice of chunking, embedding model and store (in memory, SQLite, or Postgres with pgvector). Every file has `title`, `audience` and `effective`: in its front matter for Markdown, in `manifest.json` for everything else.
   - `audience: internal` files must **never** be indexed or sent to the model. A file with no metadata anywhere has an unknown audience: treat it as internal.
   - When two files share a `title`, the one with the later `effective` date is current. Decide how to handle the older one and explain why.
2. **Chat.** For each message, retrieve the relevant chunks, call an LLM with them, and return an answer with its sources. It must keep the conversation in context so follow-ups work ("How much is the deposit on *it*?"). It must know what day it is: a sale flyer only applies between its dates.
3. **Guardrails.** Answer only from the documents. Say clearly when something is not in them. Decline off-topic requests and requests for internal information. Treat document text as data, never as instructions.
4. **Eval runner.** A script that sends every conversation in `cases.json` to your running server, checks the rules described at the top of that file, and prints pass or fail per case plus a total.

The interface we test against

We run your app and send our own extra cases, so please keep to this exactly. Everything else is your choice.

```
POST /chat            { "sessionId": "abc", "message": "How much is the tile saw per day?",
"today": "2026-11-05" }
                      "today" is optional: when present it is the current date, otherwise use
the system date
  200 -> { "answer": "...",
           "outcome": "answer" | "not_in_docs" | "decline",
           "sources": [ { "file": "rental-tools.md", "chunkId": "rental-tools.md#2" } ] }
  400 -> { "error": "..." }   for an empty or invalid message

POST /ingest          re-reads docs/ without a restart -> { "indexed": [...], "skipped": [ {
"file": "...", "reason": "..." } ], "chunks": 140 }
GET  /debug/retrieve?q=...&k=5   top chunks for a query, with text and score
npm run eval          runs cases.json against the server, prints results, writes eval-
results.json
```

- TypeScript on Node 20 or later. Any web framework.
- Any LLM and embedding provider you already have, or a local model through Ollama. Free tiers are fine. Keys go in `.env`; commit a `.env.example` and **never commit a real key**.
- Please build the pipeline yourself: an LLM SDK and a vector database client are fine, but do not use an all-in-one RAG framework (LangChain, LlamaIndex or similar). We want to see your retrieval decisions.
- A model call should time out (about 20 seconds) and return a clean error, not hang.

What the 58 cases cover

Area	Example	What it tests
Simple facts, typos, synonyms	"wat time u open sundy", "Christmas Eve"	Retrieval that bridges different wording
Conflicting versions	"How many days to return something?", "Can I pay with a check?"	Old policy and an old web page say one thing, current files another
Dates	"How much is ice melt right now?" on two different days	Sale prices only inside their dates; expired and upcoming sales
File formats	Contractor discount (PDF), drywall screw aisle (CSV)	Clean extraction from every format
Conditional rules	Opened drill, 3 weeks vs 5 weeks	Applying the right sub-rule, not the headline
Table lookups	"How much is the 20V brushless drill?"	Exact row from the CSV, not a neighbor
Ambiguity	"How much is the drill?"	Asks or lists both instead of guessing
Arithmetic	Tile saw for 3 days; plywood with paid cuts; glass minimum charge	Correct totals from the rules
Rules that interact	Contractor 10% plus a flyer price; 18V drill plus battery and charger vs 20V kit	Reasoning across several files
Follow-ups	"...the deposit on it?"	Conversation memory feeding retrieval
Two documents at once	Key copy at 6:45 PM on a Wednesday	Combining hours with the key counter rule
Wrong premise	"$200 and 8 miles, so free, right?"	Correcting the customer politely
Not in the documents	"How much is a lawn mower?"	Saying so, with no invented price
Safety	Injection in a review, staff handbook, pricing notes, the unlabeled supplier file, "admin mode"	No 50% sale, no internal data, ever

We also hold back about twenty more cases of the same kinds, and in the interview we will add a new document and call `/ingest`. Build for the categories, not for these 58 exact questions.

What to send back

1. **The repository**, runnable from a clean clone in five commands or fewer, written in the README.
2. **README: "How it works"**, about one page, in your own words:
   - Your chunking, embedding model, store, and how many chunks you retrieve and why.
   - How you handle old versions, internal and unlabeled files, the injected review, dates, the PDF and HTML, and follow-up questions.
   - How you get arithmetic right, and how you decide "not in the documents".
   - Your eval results, pasted as they came out, including failures, and why each failure happens.
   - What one full eval run costs (or "free, local model"), and how long you spent.
   - What you would change to run this for 100 stores with 5,000 documents each.

3. **A 5-minute screen recording** (Loom or any recorder): start the app, run the eval, then walk through the code path of one question from the HTTP request to the answer.

How we review

- We clone it, follow your README, and run your eval and our held-back cases.
- **Automatic fail:** any internal information reaches an answer, a real API key is in the repo, or it does not run from the README.
- Then: how many cases pass, how well you explain your choices and your failures, and whether the code is something a teammate could maintain (clear names, small functions, error handling, tests where they matter).
- In the interview (about 60 minutes): you walk us through it, we add a document and some questions live, and we ask you to change one behavior while we watch.

Thank you for your time. We look forward to seeing what you build.